# Hospital Management System

No-SQL-Based Project Documentation ,Project 4

**Team 7:**

| **Team Members** | **Roll No** |
|---|---|
| **Sandip Maity** | **UG/SOET/165/23/082** |
| **Sanskriti Sharma** | **UG/SOET/165/23/081** |
| **Arindam Jana** | **UG/02/BTCSECSF/2023/002** |
| **Sazidur Rahaman** | **UG/SOET/165/24/001** |

INDEX


| List | Topic | Page No |
|---|---|---|
| 1 | Project Overview | 3 |
| 2 | Objective | 3 |
| 3 | System Architecture | 4 |
| 4 | Data Modeling & Strategy | 5 |
| 5 | CRUD Operations | 8 |
| 6 | Aggregation Pipeline | 10 |
| 7 | Indexing | 12 |
| 8 | Setup & Installation | 13 |
| 9 | Conclusion | 14 |

## 1. PROJECT OVERVIEW

The Hospital Management System is a NoSQL-based application designed to store, manage, and analyze hospital data including patients, doctors, and appointments. The system allows booking appointments, storing patient history using embedded documents, and performing aggregation queries for analytics.

The project uses a document-oriented NoSQL database (MongoDB) to handle unstructured and semi-structured healthcare data efficiently.

Repository Structure (from GitHub HMS folder):api

- backend
- frontend
- package.json
- Vercel.json
  this indicates a full-stack implementation with frontend UI, backend API, and database integration.

## 2. OBJECTIVE

Store and analyze patient records using NoSQL database with:

- Collections: patients, doctors, appointments
- Appointment booking system
- Embedded patient history
- Aggregation analytics
- Indexing for performance

## 3. SYSTEM ARCHITECTURE

### 3.1 Technology Stack

Frontend

- HTML / CSS / JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB (NoSQL)

Deployment

- Vercel (frontend)
- Node backend API

### 3.2 collections overview

The HMS uses three primary MongoDB collections that mirror the core entities of a hospital environment:

| Collection | Purpose | Key Relationships |
|---|---|---|
| patients | Stores patient demographics and embedded medical history | References doctors via doctor_id in history |
| doctors | Stores doctor profiles and specializations | Referenced by appointments and patient history |
| appointments | Books and tracks patient-doctor appointment slots | Links patient_id ↔ doctor_id with timestamps |

## 4. DATA MODELING AND STRATEGY

The project adopts a **hybrid modeling approach** to balance read performance with data integrity.

## Collections & Schema Design

- **patients Collection (Embedding):**

  **Rationale:** Medical history is embedded directly within the patient document. This allows for a "single atomic read" to retrieve a complete clinical profile without needing JOINS.

  **Fields:** firstName, lastName, dob, bloodGroup, contact, and an array of medicalHistory.

patients

| | _id … | name String | age… | gender … | blood_typ… | contact String | medical_history… | created_at Date | email String | __v Int32 | address String |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Obje… | "Ravi Pate… | 58 | "Male" | "A-" | "9700003333" | [] 1 elements | 2026-04-05T07:… | No field | 0 | No field |
| 2 | Obje… | "Meena Kri… | 32 | "Female" | "O+" | "9700002222" | [] 1 elements | 2026-04-05T07:… | No field | 0 | No field |
| 3 | Obje… | "Ravi Pate… | 58 | "Male" | "A-" | "9700003333" | [] 1 elements | 2026-04-05T07:… | No field | 0 | No field |
| 4 | Obje… | "Sunita Bo… | 27 | "Female" | "AB+" | "9700004444" | [] 0 elements | 2026-04-05T07:… | No field | 0 | No field |
| 5 | Obje… | "Arjun Das" | 45 | "Male" | "B+" | "9700001111" | [] 2 elements | 2026-04-05T07:… | No field | 0 | No field |
| 6 | Obje… | "Meena Kri… | 32 | "Female" | "O+" | "9700002222" | [] 1 elements | 2026-04-05T07:… | No field | 0 | No field |
| 7 | Obje… | "Krrish Ya… | 22 | "Male" | "A+" | "9330422055" | [] 1 elements | 2026-04-06T09:… | "krish@24g… | 0 | "Howrah" |

- **appointments Collection (Referencing):**

  **Rationale:** Uses ObjectId references for patientId and doctorId. Since doctors and appointments have independent lifecycles, referencing prevents redundant data updates.

  **Fields:** patientId, doctorId, department, scheduledDate, status.

appointments

| | _id Objec… | patient_id… | doctor_id … | patient_name Str… | doctor_name String | date Date | time St… | reason String | status String |
|---|---|---|---|---|---|---|---|---|---|
| 1 | ObjectId(… | ObjectId(… | ObjectId(… | "Meena Krishnan" | "Dr. Rajan Mehta" | 2026-03-31T07:… | "11:30" | "Severe headache" | "completed" |
| 2 | ObjectId(… | ObjectId(… | ObjectId(… | "Meena Krishnan" | "Dr. Priya Nair" | 2026-03-28T07:… | "15:00" | "Child health consultatio… | "completed" |
| 3 | ObjectId(… | ObjectId(… | ObjectId(… | "Ravi Patel" | "Dr. Ananya Sharm… | 2026-04-02T07:… | "09:00" | "ECG and lipid profile" | "completed" |
| 4 | ObjectId(… | ObjectId(… | ObjectId(… | "Ravi Patel" | "Dr. Ananya Sharm… | 2026-03-21T07:… | "09:30" | "Post-surgery checkup" | "completed" |
| 5 | ObjectId(… | ObjectId(… | ObjectId(… | "Sunita Bose" | "Dr. Kavitha Redd… | 2026-04-06T07:… | "14:00" | "Skin rash consultation" | "scheduled" |
| 6 | ObjectId(… | ObjectId(… | ObjectId(… | "Arjun Das" | "Dr. Ananya Sharm… | 2026-03-26T07:… | "10:00" | "Chest pain follow-up" | "completed" |
| 7 | ObjectId(… | ObjectId(… | ObjectId(… | "Meena Krishnan" | "Dr. Rajan Mehta" | 2026-03-31T07:… | "11:30" | "Severe headache" | "completed" |
| 8 | ObjectId(… | ObjectId(… | ObjectId(… | "Arjun Das" | "Dr. Ananya Sharm… | 2026-04-08T07:… | "11:00" | "Monthly BP checkup" | "scheduled" |
| 9 | ObjectId(… | ObjectId(… | ObjectId(… | "Meena Krishnan" | "Dr. Priya Nair" | 2026-03-28T07:… | "15:00" | "Child health consultatio… | "completed" |

- **doctors Collection:**

  Stores provider details such as name, specialization, Department, Experience, Contact, Email, Consultation fee and available Days.

doctors

| | _id ObjectId | name String | specialization… | department Stri… | experien… | contact String | consultation… | available_days St… | created_at Date |
|---|---|---|---|---|---|---|---|---|---|
| 1 | ObjectId('69… | "Dr. Rajan Meh… | "Neurology" | "Brain & Spine" | 10 | "9800002222" | 1000 | "Tue, Thu, Sat" | 2026-04-05T07:11:4 |
| 2 | ObjectId('69… | "Dr. Priya Nai… | "Pediatrics" | "Child Health" | 8 | "9800003333" | 600 | "Mon, Tue, Wed" | 2026-04-05T07:11:4 |
| 3 | ObjectId('69… | "Dr. Sameer Ku… | "Orthopedics" | "Bone & Joint" | 12 | "9800004444" | 700 | "Wed, Fri, Sat" | 2026-04-05T07:11:4 |
| 4 | ObjectId('69… | "Dr. Kavitha R… | "Dermatology" | "Skin Care" | 6 | "9800005555" | 500 | "Mon, Thu, Sat" | 2026-04-05T07:11:4 |
| 5 | ObjectId('69… | "Dr. Ananya Sh… | "Cardiology" | "Heart Care" | 14 | "9800001111" | 800 | "Mon, Wed, Fri" | 2026-04-05T07:11:4 |
| 6 | ObjectId('69… | "Dr. Rajan Meh… | "Neurology" | "Brain & Spine" | 10 | "9800002222" | 1000 | "Tue, Thu, Sat" | 2026-04-05T07:11:4 |
| 7 | ObjectId('69… | "Dr. Priya Nai… | "Pediatrics" | "Child Health" | 8 | "9800003333" | 600 | "Mon, Tue, Wed" | 2026-04-05T07:11:4 |
| 8 | ObjectId('69… | "Dr. Sameer Ku… | "Orthopedics" | "Bone & Joint" | 12 | "9800004444" | 700 | "Wed, Fri, Sat" | 2026-04-05T07:11:4 |
| 9 | ObjectId('69… | "Dr. Kavitha R… | "Dermatology" | "Skin Care" | 6 | "9800005555" | 500 | "Mon, Thu, Sat" | 2026-04-05T07:11:4 |

ER Diagram

medical_history
id string pk
patient_id string
condition string
diagnosis_date date
treatment string
notes string
recorded_at timestamp
patients
id string pk
name string
age int
gender string
blood_type string
contact string
email string
address string
created_at timestamp
appointments
id string pk
patient_id string
doctor_id string
patient_name string
doctor_name string
date date
time string
reason string
status string
created_at timestamp
doctors
id string pk
name string
specialization string
department string
experience int
contact string
email string
consultation_fee decimal
available_days string
created_at timestamp

## 5. CRUD OPERATIONS

The system exposes a versioned **RESTful API** for all core resources. **Create Read Update Delete using JavaScript and MongoDB Atlas**

### 5.1 Create(Insert Documents)

**Insert a Doctor:**

```
const doctor = {doctor_id: "D001",
name: "Dr. Priya Sharma",
specialization: "Cardiology",
qualification: "MD, DM",
experience_years: 12,
availability: ["Monday", "Wednesday", "Friday"]};

db.doctors.insertOne(doctor);
```

**Insert a Patient with Embedded History:**

```
const patient = {
  patient_id: "P001",
  name: "Ravi Kumar",
  age: 45,
  blood_group: "B+",
  history: [{visit_date: new Date("2024-01-15"),
      doctor_id: "D001",
      diagnosis: "Hypertension",
      prescription: "Amlodipine 5mg"}]};

db.patients.insertOne(patient);
```

**Book an Appointment:**

```
const appointment = {
  appointment_id: "A001",
  patient_id: "P001",
  doctor_id: "D001",
  appointment_date: new Date("2024-09-10"),
  time_slot: "10:00 AM",
  status: "Confirmed"};
```

```
db.appointments.insertOne(appointment);
```

**5.2 Read (Query Documents)**

**Find All Patients**

```
const patients = db.patients.find();
patients.forEach(p => { print(p.name, p.age);});
```

**Find Patients by Doctor in History**

```
db.patients.find({ "history.doctor_id": "D001" },{ name: 1, "history.$": 1 });
```

**Find Upcoming Appointments**

```
db.appointments.find({doctor_id: "D001", appointment_date: { $gte: new Date() },
status: "Confirmed"});
```

**5.3 Update (Modify Documents)**

**Update Appointment Status**

```
db.appointments.updateOne( { appointment_id: "A001" }, { $set:
{ status:Completed" } });
```

Append a New Visit to Patient History

```
const newVisit = { visit_date: new Date("2024-11-05"),  doctor_id: "D002",
diagnosis: "Seasonal Flu",  prescription: "Paracetamol 500mg"};

db.patients.updateOne({ patient_id: "P001" }, { $push: { history: newVisit }});
```

**5.4 Delete (Remove Documents)**

**Cancel and Remove an Appointment**

```
db.appointments.deleteOne({ appointment_id: "A001"});
```

**Remove a Patient Record**

```
db.patients.deleteOne({ patient_id: "P001"});
```

**Add patient**
db.patients.insertOne({...})

**Add doctor**
db.doctors.insertOne({...})

**Book appointment**
db.appointments.insertOne({...})

**Read**
**Get all doctors**

db.doctors.find()

**Get patient history**
db.patients.find({ patient_id: "P101" })

**Update**

**Update patient history**
db.patients.updateOne(...)

**Update appointment**
db.appointments.updateOne(...)

**Delete**

**Cancel appointment**
db.appointments.deleteOne({ appointment_id: "A101"})

## 6. AGGREGATION PIPELINES

To meet the requirement for doctor-wise analytics, the system utilizes the MongoDB Aggregation Pipeline. This processes data within the database engine to reduce network overhead.

- **Patients per Doctor:** Uses $match, $group, and $sort stages to calculate current workloads.
- **Most Visited Doctor:** Aggregates appointment counts grouped by doctorId to identify high-demand providers.

### 6.1 Number of Patients Per Doctor

```
const pipelinePatientsPerDoctor = [{ $unwind: "$history"},
  { $group: { _id: "$history.doctor_id", total_patients: { $sum: 1 }}},
  { $sort: { total_patients: -1 }},
  { $lookup: { from: "doctors", localField: "_id", foreignField: "doctor_id", as:
"doctor_info" }},
  { $project: { doctor_id: "$_id", doctor_name: { $arrayElemAt:
["$doctor_info.name", 0] },
     total_patients: 1, _id: 0  }}];

const result = db.patients.aggregate(pipelinePatientsPerDoctor);
```

**Sample Output**

```
[ { doctor_id: "D001", doctor_name: "Dr. Priya Sharma", total_patients: 8 },
 { doctor_id: "D002", doctor_name: "Dr. Arjun Mehta", total_patients: 6 },
{ doctor_id: "D003", doctor_name: "Dr. Sunita Roy", total_patients: 4 }]
```

### 6.2 Most Visited Doctor

**Counts appointments → sorts → returns highest visited doctor.**

```
constpipelineMostVisited= [{
   $group: {
     _id: "$doctor_id",
     visit_count: { $sum: 1 }}},
  {$sort: { visit_count: -1 }},
  { $limit: 1},
  {$lookup: {
     from: "doctors",
     localField: "_id",
     foreignField: "doctor_id",
     as: "doctor_info"}},
```

```
  { $project: {doctor_id: "$_id",
     doctor_name: { $arrayElemAt: ["$doctor_info.name", 0] },
     specialization: { $arrayElemAt: ["$doctor_info.specialization", 0] },
     visit_count: 1,
     _id: 0  }}];

constmostVisited=db.appointments.aggregate(pipelineMostVisited);
```

**Sample Output**

```
[{ doctor_id: "D001",
  doctor_name: "Dr. Priya Sharma",
  specialization: "Cardiology",
  visit_count: 12 }]
```

## 7. INDEXING

Indexing improves performance for frequently queried fields like doctor_id, patient_id, and dates.

### 7.1 Create Indexes

**Index on doctor_id (appointments)**

```
db.appointments.createIndex(
  { doctor_id: 1 },
  { name: "idx_doctor_id" });
```

Compound Index (doctor_id + appointment_date)

```
db.appointments.createIndex({ doctor_id: 1, appointment_date: -1 },
  { name: "idx_doctor_date" });
```

Index on patient_id

```
db.appointments.createIndex(
  { patient_id: 1 },
  { name: "idx_patient_id" });
```

Index on Embedded Field (history.doctor_id)

```
db.patients.createIndex(
{ "history.doctor_id": 1 },
{ name: "idx_history_doctor_id" });
```

### 7.2 Verifying Indexes

**List All Indexes**

```
db.appointments.getIndexes();
```

**Verify Index Usage using explain()**

```
const explainResult = db.appointments.find(
  { doctor_id: "D001" }
).explain("executionStats");

print(explainResult.executionStats.totalDocsExamined);
```

## 8. SETUP & INSTALLATION

### 8.1 Prerequisites

- Node.js (v16 or above)
- MongoDB 6.x (Local installation or MongoDB Atlas)
- npm (comes with Node.js)
- Git & GitHub

**Deployment:**
Vercel account

### 8.2 Environment Variables

Create a .env file inside the **backend** directory:

MONGO_URI=mongodb://127.0.0.1:27017
MONGO_DB_NAME=medcore_hms
PORT=3000

**For MongoDB Atlas:**

MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/
MONGO_DB_NAME=medcore_hms

### 8.3 Running the Project

**Step 1: Start MongoDB Service (Local)**

**Linux:**
sudo systemctl start mongod

**Windows:**
net start MongoDB

**Mac**:
brew services start mongodb-community

**Step 2: Clone Repository**

git clone https://github.com/Sandip-Maity-2023/Unstructured_Database_Lab.git

**Step 3: Navigate to HMS Project**

cd Unstructured_Database_Lab/HMS

**Step 4: Install Dependencies**

npm install

This installs:

- express
- mongoose
- cors
- dotenv (if used)

**Step 5: Start Backend Server**

node server.js

or (depending on file name)

Server will start at:

http://localhost:3000

**Step 6: Open Application**

Open browser:

http://localhost:3000

Frontend loads automatically from:

frontend/index.html

**Step 7: Verify MongoDB Collections**

Open MongoDB shell:

mongosh

Switch database:

use medcore_hms

Show collections:

show collections

Expected output:

patients
doctors
appointments

### 8.4 API Health Check

http://localhost:3000/api/health

Expected Response:

```
{
"status": "ok",
"database": "connected"
}
```

### 8.5 Seed Data

When server starts, sample data is automatically inserted:

Doctors:

- Cardiology
- Neurology
- Pediatrics
- Orthopedics
- Dermatology

Patients:

- 5 sample patients

Appointments:

- Scheduled + completed

## 9. CONCLUSION

The Hospital Management System using MongoDB provides an efficient NoSQL solution for managing healthcare data. The system uses three main collections—patients, doctors, and appointments—with embedded patient history to support flexible data storage. It implements complete CRUD operations, aggregation queries for doctor-wise analytics, and indexing for improved performance. Overall, the project successfully demonstrates data modeling, data management, and analytics using MongoDB in a scalable and practical healthcare application.